

Food Store - Sistema de Gestión de Pedidos de Comida

Institución:

UTN - Facultad Regional San Nicolás

Carrera:

Tecnicatura Universitaria en Programación a Distancia

Materia:

Programación 2

Título del proyecto:

Food Store - Sistema de Gestión de Pedidos de Comida

Estudiante:

Carolina Rosario Calonico

Fecha:

Junio 2026

Índice

1. Introducción
2. Objetivo del sistema
3. Marco teórico
4. Arquitectura del proyecto
5. Modelo de clases y relaciones
6. Funcionalidades implementadas
7. Validaciones y manejo de errores
8. Pruebas realizadas
9. Dificultades y soluciones
10. Conclusión
11. Bibliografía / Webgrafía
12. Links del proyecto

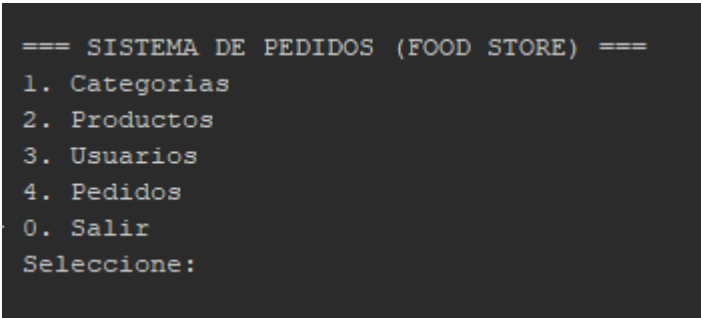
1. Introducción

El presente trabajo práctico integrador consiste en el desarrollo de una aplicación de consola denominada **Food Store**, orientada a la gestión de pedidos de comida.

El sistema fue desarrollado en lenguaje Java aplicando conceptos de Programación Orientada a Objetos. La aplicación permite administrar categorías, productos, usuarios y pedidos mediante operaciones CRUD desde un menú interactivo por consola.

La información se almacena en memoria durante la ejecución del programa utilizando colecciones, sin conexión a una base de datos. El proyecto busca representar un sistema simple pero organizado, con separación de responsabilidades entre entidades, servicios, excepciones y menú de usuario.

[CAPTURA 1 - Menú principal del sistema]



```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===  
1. Categorías  
2. Productos  
3. Usuarios  
4. Pedidos  
0. Salir  
Seleccione:
```

2. Objetivo del sistema

El objetivo principal del sistema es permitir la gestión básica de un negocio de comidas mediante una aplicación de consola.

El sistema permite realizar las siguientes operaciones:

- Gestionar categorías de productos.
- Gestionar productos asociados a una categoría.
- Gestionar usuarios.
- Registrar pedidos con uno o más detalles.
- Calcular subtotales y total del pedido.
- Actualizar estado y forma de pago de un pedido.
- Aplicar baja lógica sobre las entidades.
- Validar datos ingresados por el usuario.
- Manejar errores mediante excepciones propias.

3. Marco teórico

Programación Orientada a Objetos

La Programación Orientada a Objetos es un paradigma de programación basado en la representación de entidades mediante clases y objetos. En este proyecto se utilizaron clases para modelar las entidades principales del sistema, como `Categoria`, `Producto`, `Usuario`, `Pedido` y `DetallePedido`.

Encapsulamiento

El encapsulamiento permite proteger los atributos de una clase y acceder a ellos mediante métodos públicos. En el proyecto, los atributos de las entidades se declararon como privados y se accede a ellos mediante métodos getters y setters.

Herencia

La herencia permite reutilizar atributos y comportamientos comunes entre varias clases. En este sistema se implementó una clase abstracta `Base`, de la cual heredan las entidades principales. Esta clase contiene atributos comunes como `id`, `eliminado` y `createdAt`.

Interfaces

Las interfaces permiten definir comportamientos comunes que luego son implementados por distintas clases. En el proyecto se creó la interfaz `Calculable`, que define el método `calcularTotal()`. La clase `Pedido` implementa esta interfaz para calcular el total del pedido.

Colecciones

Las colecciones permiten almacenar y administrar conjuntos de objetos en memoria. En este proyecto se utilizaron listas dinámicas para guardar categorías, productos, usuarios y pedidos durante la ejecución del programa.

Enumeraciones

Las enumeraciones permiten definir un conjunto fijo de valores posibles. En el sistema se utilizaron enums para representar el rol del usuario, el estado del pedido y la forma de pago.

Excepciones propias

El manejo de excepciones permite controlar errores sin que el programa finalice de manera inesperada. En el proyecto se implementaron excepciones propias para representar situaciones como datos inválidos, entidades no encontradas, duplicados y stock insuficiente.

4. Arquitectura del proyecto

El proyecto se organizó utilizando una estructura por paquetes, con el objetivo de separar responsabilidades y mejorar la mantenibilidad del código.

La estructura principal es la siguiente:

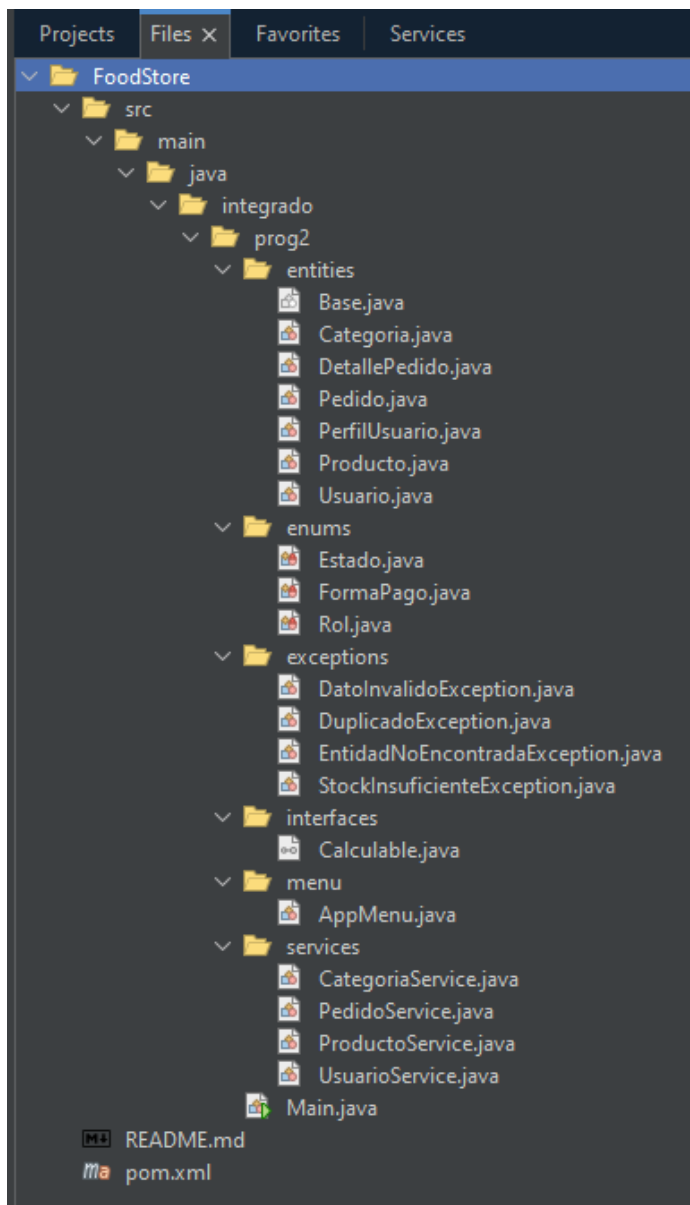
```
src/main/java/integrado/prog2
├── Main.java
├── entities
├── enums
├── exceptions
├── interfaces
├── menu
└── services
```

Descripción de paquetes

- `entities`: contiene las clases del modelo de dominio.
- `enums`: contiene las enumeraciones utilizadas en el sistema.
- `exceptions`: contiene las excepciones propias.
- `interfaces`: contiene la interfaz `Calculable`.
- `services`: contiene la lógica de negocio y las colecciones en memoria.
- `menu`: contiene la interacción con el usuario por consola.
- `Main.java`: punto de entrada de la aplicación.

Esta organización permite que el menú se encargue principalmente de la interacción con el usuario, mientras que las reglas de negocio y validaciones se concentran en los servicios.

[CAPTURA 2 - Estructura del proyecto en NetBeans]



5. Modelo de clases y relaciones

El sistema se diseñó utilizando clases que representan las entidades principales del dominio.

Clase Base

La clase Base es una clase abstracta que contiene atributos comunes a las entidades:

- id
- eliminado
- createdAt

Todas las entidades principales heredan de esta clase.

Categoría

La clase `Categoria` representa una categoría de productos. Contiene nombre y descripción. Una categoría puede estar asociada a varios productos.

Producto

La clase `Producto` representa un producto del catálogo. Contiene nombre, precio, descripción, stock, imagen, disponibilidad y una categoría asociada.

Usuario

La clase `Usuario` representa a un usuario del sistema. Contiene nombre, apellido, mail, celular, contraseña y rol. Además, se implementó una relación uno a uno entre `Usuario` y `PerfilUsuario`.

PerfilUsuario

La clase `PerfilUsuario` contiene información complementaria del usuario, como dirección y observaciones. Esta clase permite representar una relación uno a uno real dentro del código.

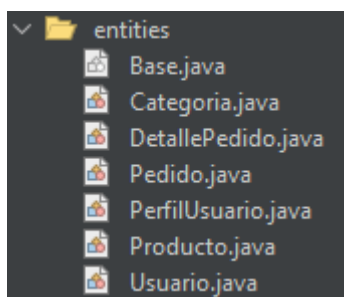
Pedido

La clase `Pedido` representa una compra realizada por un usuario. Contiene fecha, estado, total, forma de pago, usuario asociado y una lista de detalles. Además, implementa la interfaz `Calculable`.

DetallePedido

La clase `DetallePedido` representa cada producto incluido dentro de un pedido. Contiene cantidad, subtotal y producto asociado.

[CAPTURA 3 - Paquete entities con las clases principales]



6. Funcionalidades implementadas

Gestión de categorías

El sistema permite:

- Listar categorías activas.

- Crear categorías.
- Editar categorías existentes.
- Eliminar categorías mediante baja lógica.
- Validar nombres repetidos.
- Validar que el nombre ingresado tenga un formato correcto.
-

[CAPTURA 4 - Listado de categorías]

```

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 1

--- CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
ID: 1 | Categoría: Hamburguesas | Descripción: Comidas rapidas con pan y medallon
ID: 2 | Categoría: Bebidas | Descripción: Gaseosas, aguas y jugos

```

Gestión de productos

El sistema permite:

- Listar productos activos.
- Mostrar la descripción del producto en los listados.
- Crear productos asociados a una categoría.
- Editar productos.
- Eliminar productos mediante baja lógica.
- Validar precio y stock.
- Validar nombre único.
- Validar que la categoría asociada exista y esté activa.
-

[CAPTURA 5 - Listado de productos]

```

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 2

--- PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
ID: 1 | Producto: Hamburguesa clasica | Descripcion: Hamburguesa con queso | Precio: $4500.0 | Stock: 10 | Categoria: Hamburguesas | Disponible: Si
ID: 2 | Producto: Agua mineral | Descripcion: Botella 500 ml | Precio: $1200.0 | Stock: 20 | Categoria: Bebidas | Disponible: Si

```

[CAPTURA 6 - Creación de un producto]

```

--- PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Jugo de naranja
Descripcion: Exprimido
Precio: 900
Stock: 10
Imagen/URL: img.jpg
Esta disponible? (S/N): S
ID: 1 | Categoria: Hamburguesas | Descripcion: Comidas rapidas con pan y medallon
ID: 2 | Categoria: Bebidas | Descripcion: Gaseosas, aguas y jugos
ID de categoria: 2
Producto creado correctamente. ID generado: 3

--- PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
ID: 1 | Producto: Hamburguesa clasica | Descripcion: Hamburguesa con queso | Precio: $4500.0 | Stock: 10 | Categoria: Hamburguesas | Disponible: Si
ID: 2 | Producto: Agua mineral | Descripcion: Botella 500 ml | Precio: $1200.0 | Stock: 20 | Categoria: Bebidas | Disponible: Si
ID: 3 | Producto: Jugo de naranja | Descripcion: Exprimido | Precio: $900.0 | Stock: 10 | Categoria: Bebidas | Disponible: Si

```

Gestión de usuarios

El sistema permite:

- Listar usuarios activos.
- Crear usuarios.
- Editar usuarios.
- Eliminar usuarios mediante baja lógica.
- Validar mail único.
- Validar formato básico de los datos ingresados.
- Asociar cada usuario a un perfil mediante una relación uno a uno.

[CAPTURA 7 - Listado de usuarios]

```

--- USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
ID: 1 | Usuario: Ana Gomez | Mail: ana@mail.com | Celular: 1122334455 | Rol: USUARIO | Direccion: Av. Siempre Viva 123

```

Gestión de pedidos

El sistema permite:

- Listar pedidos activos.
- Crear pedidos con detalles.
- Seleccionar un usuario activo.
- Seleccionar productos activos.
- Ingresar cantidades.
- Calcular subtotales por detalle.
- Calcular el total del pedido.
- Actualizar estado y forma de pago.
- Eliminar pedidos mediante baja lógica.

[CAPTURA 8 - Creación de pedido con detalles]

```

--- PEDIDOS ---
1. Listar
2. Crear pedido con detalles
3. Actualizar estado/forma de pago
4. Eliminar
0. Volver
Seleccione: 2
ID: 1 | Usuario: Ana Gomez | Mail: ana@mail.com | Celular: 1122334455 | Rol: USUARIO | Direccion: Av. Siempre Viva 123
ID de usuario: 1
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
Seleccione forma de pago: 3
ID: 1 | Producto: Hamburguesa clasica | Descripcion: Hamburguesa con queso | Precio: $4500.0 | Stock: 10 | Categoria: Hamburguesas | Disponible: Si
ID: 2 | Producto: Agua mineral | Descripcion: Botella 500 ml | Precio: $1200.0 | Stock: 20 | Categoria: Bebidas | Disponible: Si
ID: 3 | Producto: Jugo de naranja | Descripcion: Exprimido | Precio: $900.0 | Stock: 10 | Categoria: Bebidas | Disponible: Si
ID de producto: 1
Cantidad: 2
Detalle agregado correctamente.
Agregar otro producto? (S/N): N
Pedido creado correctamente. ID generado: 1 | Total: $9000.0

```

[CAPTURA 9 - Listado de pedidos con total]

```

--- PEDIDOS ---
1. Listar
2. Crear pedido con detalles
3. Actualizar estado/forma de pago
4. Eliminar
0. Volver
Seleccione: 1
ID: 1 | Usuario: Ana Gomez | Estado: PENDIENTE | Pago: EFECTIVO | Total: $9000.0 | Fecha: 2026-06-16
- Detalle ID: 1 | Producto: Hamburguesa clasica | Cantidad: 2 | Subtotal: $9000.0

```

7. Validaciones y manejo de errores

El sistema implementa validaciones para evitar datos incorrectos o inconsistentes.

Algunas de las validaciones aplicadas son:

- No permitir categorías con nombres repetidos.
- No permitir productos con nombres repetidos.
- No permitir productos con precio o stock negativo.
- No permitir usuarios con mail repetido.
- No permitir pedidos sin usuario.
- No permitir detalles de pedido con cantidad menor o igual a cero.

- No permitir agregar productos inexistentes o eliminados a un pedido.
- No permitir superar el stock disponible.
- Validar IDs inexistentes antes de continuar con una operación.
- Cancelar la creación de un pedido si ocurre un error durante la carga de detalles.

Para controlar estos casos se utilizaron excepciones propias:

- DatoInvalidoException
- DuplicadoException
- EntidadNoEncontradaException
- StockInsuficienteException

Estas excepciones son capturadas desde el menú para mostrar mensajes claros al usuario y evitar que el programa finalice inesperadamente.

[CAPTURA 10 - Ejemplo de validación o mensaje de error]

```

--- CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
ID: 1 | Categoria: Hamburguesas | Descripcion: Comidas rapidas con pan y medallon
ID: 2 | Categoria: Bebidas | Descripcion: Gaseosas, aguas y jugos

--- CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Bebidas
Error: Ya existe una categoria con ese nombre.

```

8. Pruebas realizadas

Se realizaron pruebas manuales desde la consola para verificar el correcto funcionamiento del sistema.

Pruebas sobre categorías

- Listar categorías cargadas.
- Crear una nueva categoría.
- Intentar crear una categoría repetida.
- Editar una categoría existente.
- Intentar editar una categoría inexistente.

- Eliminar una categoría mediante baja lógica.

Pruebas sobre productos

- Listar productos cargados.
- Crear un producto válido.
- Intentar crear un producto con nombre repetido.
- Intentar crear un producto con precio negativo.
- Intentar crear un producto con stock negativo.
- Editar un producto existente.
- Intentar eliminar un producto inexistente.

Pruebas sobre usuarios

- Listar usuarios cargados.
- Crear un usuario válido.
- Intentar crear un usuario con mail repetido.
- Intentar crear un usuario con mail inválido.
- Editar un usuario existente.
- Eliminar un usuario mediante baja lógica.

Pruebas sobre pedidos

- Crear un pedido con un usuario activo.
- Agregar productos al pedido.
- Validar cálculo de subtotal y total.
- Intentar agregar un producto inexistente.
- Intentar agregar una cantidad mayor al stock.
- Verificar que ante errores se cancele la creación del pedido.
- Verificar que el stock se mantenga consistente si se cancela el pedido.
- Actualizar estado y forma de pago.
- Eliminar pedido mediante baja lógica.

[CAPTURA 11 - Prueba de error controlado durante la creación de pedido]

```

--- PEDIDOS ---
1. Listar
2. Crear pedido con detalles
3. Actualizar estado/forma de pago
4. Eliminar
0. Volver
Seleccione: 2
ID: 1 | Usuario: Ana Gomez | Mail: ana@mail.com | Celular: 1122334455 | Rol: USUARIO | Direccion: Av. Siempre Viva 123
ID de usuario: 1
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
Seleccione forma de pago: 1
ID: 1 | Producto: Hamburguesa clasica | Descripcion: Hamburguesa con queso | Precio: $4500.0 | Stock: 10 | Categoria: Hamburguesas | Disponible: Si
ID: 2 | Producto: Agua mineral | Descripcion: Botella 500 ml | Precio: $1200.0 | Stock: 20 | Categoria: Bebidas | Disponible: Si
ID de producto: 1
Cantidad: 2
Detalle agregado correctamente.
Agregar otro producto? (S/N): S
ID: 1 | Producto: Hamburguesa clasica | Descripcion: Hamburguesa con queso | Precio: $4500.0 | Stock: 8 | Categoria: Hamburguesas | Disponible: Si
ID: 2 | Producto: Agua mineral | Descripcion: Botella 500 ml | Precio: $1200.0 | Stock: 20 | Categoria: Bebidas | Disponible: Si
ID de producto: 99
Error: No se encontro un producto activo con ID 99.
La creacion del pedido fue cancelada para evitar datos inconsistentes.

```

[CAPTURA 12 - Verificación de stock luego de cancelar un pedido con error]

```

--- PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
ID: 1 | Producto: Hamburguesa clasica | Descripcion: Hamburguesa con queso | Precio: $4500.0 | Stock: 10 | Categoria: Hamburguesas | Disponible: Si
ID: 2 | Producto: Agua mineral | Descripcion: Botella 500 ml | Precio: $1200.0 | Stock: 20 | Categoria: Bebidas | Disponible: Si

```

9. Dificultades y soluciones

Durante el desarrollo surgieron algunas dificultades relacionadas con la validación de datos y la consistencia de la información almacenada en memoria.

Una de las principales dificultades fue controlar que las operaciones de creación de pedidos no dejaran datos inconsistentes. Por ejemplo, si se agregaba un producto al pedido y luego ocurría un error con otro producto, era necesario cancelar la creación del pedido y devolver el stock descontado previamente.

Para solucionar esto, se implementó una lógica de cancelación segura, evitando que el pedido se guarde en memoria si ocurre una excepción durante la carga de detalles.

Otra dificultad fue organizar el código para que el menú no concentrara toda la lógica del sistema. Para resolverlo, se separaron las responsabilidades en servicios, entidades, excepciones y menú.

También se agregaron validaciones progresivas para que el sistema informe los errores apenas se ingresan datos inválidos, evitando que el usuario continúe cargando información innecesaria.

10. Conclusión

El proyecto Food Store permitió integrar los principales conceptos vistos en Programación 2 mediante una aplicación de consola desarrollada en Java.

Se aplicaron conceptos de Programación Orientada a Objetos, herencia, encapsulamiento, interfaces, enumeraciones, colecciones y manejo de excepciones. Además, se implementó una arquitectura separada por paquetes, lo que facilita la organización y mantenimiento del código.

El sistema cumple con las funcionalidades principales solicitadas: CRUD de categorías, productos, usuarios y pedidos, baja lógica, validaciones de entrada, manejo de errores y cálculo de totales en pedidos.

11. Bibliografía / Webgrafía

- Documentación oficial de Java: <https://docs.oracle.com/en/java/>
- Documentación de Maven: <https://maven.apache.org/>
- Material de la materia Programación 2.
- Consigna del Trabajo Práctico Integrador.
- Pautas de corrección del Trabajo Práctico Integrador.

12. Links del proyecto

Repositorio GitHub:

<https://github.com/CarolinaCalonico/food-store-tpi-programacion-2.git>

Video demostrativo:

<https://youtu.be/JplSQpLW3U8>

Informe PDF:

El presente documento corresponde al informe técnico del proyecto.